```
#include <iostream>
#include <string>
#include <map>
#include <iomanip>
#include <stdexcept>
using namespace std;

// Constants defined in one place
constexpr long long PAISE_PER_RUPEE = 100;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int STUDENT_CONCESSION_PERCENT = 50;
constexpr int SENIOR_CONCESSION_PERCENT = 50;

constexpr int MORNING_PEAK_START = 8 * 60;
constexpr int MORNING_PEAK_END = 10 * 60;
constexpr int EVENING_PEAK_START = 17 * 60;
constexpr int EVENING_PEAK_END = 20 * 60;

constexpr long long MINIMUM_BALANCE_PAISE = 50 * PAISE_PER_RUPEE;

// Zone enumeration
enum class Zone {
    Inner,
    Middle,
    Outer
};

// Passenger base class
class Passenger {
protected:
```

```
    string passengerId;

public:
    Passenger(string id) : passengerId(id) {}

    virtual int getConcession() const {
        return 0;
    }

    virtual string getType() const {
        return "Regular";
    }

    virtual ~Passenger() = default;
};

// Student class
class Student : public Passenger {
public:
    Student(string id) : Passenger(id) {}

    int getConcession() const override {
        return STUDENT_CONCESSION_PERCENT;
    }

    string getType() const override {
        return "Student";
    }
};
```

```
// Senior citizen class
class SeniorCitizen : public Passenger {
public:
  SeniorCitizen(string id) : Passenger(id) {}

  int getConcession() const override {
    return SENIOR_CONCESSION_PERCENT;
  }

  string getType() const override {
    return "Senior Citizen";
  }
};

// Fare table class
class FareTable {
private:
  map<Zone, long long> fares;

public:
  FareTable() {
    fares[Zone::Inner] = 10 * PAISE_PER_RUPEE;
    fares[Zone::Middle] = 20 * PAISE_PER_RUPEE;
    fares[Zone::Outer] = 30 * PAISE_PER_RUPEE;
  }

  long long getFare(Zone zone) const {
    auto it = fares.find(zone);

    if (it == fares.end()) {
```

```
            throw invalid_argument("Invalid zone");
        }

        return it->second;
    }
};

// Fare calculator class
class FareCalculator {
private:
    const FareTable& fareTable;

    bool isPeakHour(int hour, int minute) const {
        if (hour < 0 || hour > 23 ||
            minute < 0 || minute > 59) {
            throw invalid_argument("Invalid time");
        }

        int time = hour * 60 + minute;

        return (time >= MORNING_PEAK_START &&
                time < MORNING_PEAK_END) ||
               (time >= EVENING_PEAK_START &&
                time < EVENING_PEAK_END);
    }

public:
    FareCalculator(const FareTable& table)
        : fareTable(table) {}
```

```
    long long calculateFare(
        Zone zone,
        const Passenger& passenger,
        int hour,
        int minute
    ) const {
        long long fare = fareTable.getFare(zone);

        if (isPeakHour(hour, minute)) {
            fare = fare * (100 + PEAK_SURCHARGE_PERCENT) / 100;
        }

        int concession = passenger.getConcession();

        fare = fare * (100 - concession) / 100;

        return fare;
    }
};

// Smart card class
class SmartCard {
private:
    string cardId;
    long long balancePaise;

public:
    SmartCard(string id, long long balance)
        : cardId(id), balancePaise(balance) {
        if (balance < 0) {
```

```
            throw invalid_argument("Invalid initial balance");
        }
    }

    long long getBalance() const {
        return balancePaise;
    }

    bool deductFare(long long fare) {
        if (fare < 0 ||
            balancePaise - fare < MINIMUM_BALANCE_PAISE) {
            return false;
        }

        balancePaise -= fare;
        return true;
    }

    string getCardId() const {
        return cardId;
    }
};

int main() {
    try {
        FareTable fareTable;
        FareCalculator calculator(fareTable);

        SmartCard card("CARD101", 100 * PAISE_PER_RUPEE);
        Student passenger("STU101");
```

```
Zone selectedZone = Zone::Middle;
int hour = 8;
int minute = 30;

long long fare = calculator.calculateFare(
   selectedZone, passenger, hour, minute
);

cout << "NAMMA METRO SMART-CARD FARE SYSTEM\n";
cout << "Card ID: " << card.getCardId() << '\n';
cout << "Passenger Type: " << passenger.getType() << '\n';
cout << "Initial Balance: Rs. "
    << fixed << setprecision(2)
    << card.getBalance() / 100.0 << '\n';

cout << "Peak Surcharge: "
    << PEAK_SURCHARGE_PERCENT << "%\n";
cout << "Student Concession: "
    << STUDENT_CONCESSION_PERCENT << "%\n";

cout << "Calculated Fare: Rs. "
    << fare / 100.0 << '\n';

if (card.deductFare(fare)) {
   cout << "Journey successful.\n";
} else {
   cout << "Journey rejected: minimum balance "
        << "of Rs. 50 must remain.\n";
}
```

```
        cout << "Remaining Balance: Rs. "
             << card.getBalance() / 100.0 << '\n';
    }
    catch (const exception& e) {
        cerr << "Error: " << e.what() << '\n';
        return 1;
    }

    return 0;
}
```